SunSwap2.0 接口文档

—— Sun 团队

v 1.0.0

目录

后端 API **3**
1.1 获取所有交易对 3
二、智能合约接口 **4**
2.1 合约地址 4
2.2 合约接口列表 4
2.2.1 TRC20 Token 4
2.2.2 Factory 4
2.2.3 ISunswapV2Router 5
2.3 合约接口详细描述 7
2.3.1 Factory 7
2.3.1.1 查询型接口 7
1、getPair 7
2、allPairs 7
3、allPairsLength 8
2.3.1.2 修改型接口 8
1、createPair 8
2.3.1.3 合约事件 9
1、PairCreated 9
2.3.2 Router 9
2.3.2.1 查询型接口 9
1、factory 9
2、WETH 9
2.3.2.2 修改型接口 10
1、addLiquidity 10
2、addLiquidityETH 11
3、removeLiquidity 12
4、removeLiquidityETH 13
5、removeLiquidityWithPermit 14
6、removeLiquidityETHWithPermit 15
7、removeLiquidityETHSupportingFeeOnTransferTokens 16
8、removeLiquidityETHWithPermitSupportingFeeOnTransferTokens 16
9、swapExactTokensForTokens 17
10、swapExactETHForTokens 18
11、swapTokensForExactETH 19
12、swapExactTokensForETH 20
13、swapETHForExactTokens 21
14、swapExactTokensForTokensSupportingFeeOnTransferTokens 22
15、swapExactETHForTokensSupportingFeeOnTransferTokens 22
16、swapExactTokensForETHSupportingFeeOnTransferTokens 23

I. 后端 API

1.1 获取所有交易对

GET: https://pabc.ablesdxd.link/swapv2/allpairs

参数:

page_size : int, 每页的条目数, 最大 50
page_num: int, 页数, 从 0 开始

返回格式:


```
{
        "data": [{
"TNUC9Qb1rRpS5CbWLmNMxXBjyFoydXjWFR_TR7NHqjeKQxGTCi8q8ZY4pL8otSzgjLj6tf":  //key: base_token 与 quote_token的id
 {
                "quote_id": "TNUC9Qb1rRpS5CbWLmNMxXBjyFoydXjWFR",
                "quote_decimal": "6", //quote_token精度
                "quote_name": "Wrapped TRX", //quote_token名称
                "quote_symbol": "WTRX", //quote_token符号
                "base_id": "TR7NHqjeKQxGTCi8q8ZY4pL8otSzgjLj6t", //base_token地址
                "base_decimal": "6", //base_token精度
                "base_name": "Tether USD", //base_token名称
                "base_symbol": "USDT", //base_token符号
                "price": "0.938196997790940827", //quote_token的价格，以base_token计价
                "quote_volume": "0", //24小时内的quote_token交易量(最小单位)
                "base_volume": "0" //24小时内的base_token交易量(最小单位)
                }
        }],
        "err_msg": "",
        "err_no": 0,
        "total_num": 143 //总条目数量
}
```

二、智能合约接口

2.1 合约地址

合约名称	合约地址	备注
factory合约	TKWJdrQkqHisa1X8HUdHEfREvTzw4pMAaY	factory 合约负责创建交易对, 和维护交易对列表
router 合约	TKzxdSv2FZKQrEqkKVgp5DcwEXBEKMg2Ax	
pair 合约	TFGDbUyP8xez44C76fin3bn3Ss6jugoUwJ	每种交易对有一个 pair 合约。 这是USDT/TRX 的pair 合约地址。

2.2 合约接口列表

2.2.1 TRC20 Token

```
interface ITRC20 {
  function transfer(address to, uint256 value) external returns (bool);
  function approve(address spender, uint256 value) external returns (bool);
  function transferFrom(address from, address to, uint256 value) external returns (bool);
  function totalSupply() external view returns (uint256);
  function balanceOf(address who) external view returns (uint256);
  function allowance(address owner, address spender) external view returns (uint256);
  event Transfer(address indexed from, address indexed to, uint256 value);
  event Approval(address indexed owner, address indexed spender, uint256 value);
}
```

2.2.2 Factory

```
interface ISunswapV2Factory {
    event PairCreated(address indexed token0, address indexed token1, address pair,
uint);
```

```
    function feeTo() external view returns (address);
    function feeToSetter() external view returns (address);

    function getPair(address tokenA, address tokenB) external view returns (address pair);
    function allPairs(uint) external view returns (address pair);
    function allPairsLength() external view returns (uint);

    function createPair(address tokenA, address tokenB) external returns (address pair);

    function setFeeTo(address) external;
    function setFeeToSetter(address) external;
}
```

2.2.3 ISunswapV2Router

```
interface ISunswapV2Router01 {
    function factory() external pure returns (address);
    function WETH() external pure returns (address);

    function addLiquidity(
        address tokenA,
        address tokenB,
        uint amountADesired,
        uint amountBDesired,
        uint amountAMin,
        uint amountBMin,
        address to,
        uint deadline
    ) external returns (uint amountA, uint amountB, uint liquidity);
    function addLiquidityETH(
        address token,
        uint amountTokenDesired,
        uint amountTokenMin,
        uint amountETHMin,
        address to,
        uint deadline
    ) external payable returns (uint amountToken, uint amountETH, uint liquidity);
    function removeLiquidity(
        address tokenA,
        address tokenB,
        uint liquidity,
        uint amountAMin,
        uint amountBMin,
        address to,
        uint deadline
    ) external returns (uint amountA, uint amountB);
```

```
    function removeLiquidityETH(
        address token,
        uint liquidity,
        uint amountTokenMin,
        uint amountETHMin,
        address to,
        uint deadline
    ) external returns (uint amountToken, uint amountETH);
    function removeLiquidityWithPermit(
        address tokenA,
        address tokenB,
        uint liquidity,
        uint amountAMin,
        uint amountBMin,
        address to,
        uint deadline,
        bool approveMax, uint8 v, bytes32 r, bytes32 s
    ) external returns (uint amountA, uint amountB);
    function removeLiquidityETHWithPermit(
        address token,
        uint liquidity,
        uint amountTokenMin,
        uint amountETHMin,
        address to,
        uint deadline,
        bool approveMax, uint8 v, bytes32 r, bytes32 s
    ) external returns (uint amountToken, uint amountETH);
    function swapExactTokensForTokens(
        uint amountIn,
        uint amountOutMin,
        address[] calldata path,
        address to,
        uint deadline
    ) external returns (uint[] memory amounts);
    function swapTokensForExactTokens(
        uint amountOut,
        uint amountInMax,
        address[] calldata path,
        address to,
        uint deadline
    ) external returns (uint[] memory amounts);
    function swapExactETHForTokens(uint amountOutMin, address[] calldata path, address
to, uint deadline)
        external
        payable
        returns (uint[] memory amounts);
    function swapTokensForExactETH(uint amountOut, uint amountInMax, address[]
calldata path, address to, uint deadline)
        external
        returns (uint[] memory amounts);
    function swapExactTokensForETH(uint amountIn, uint amountOutMin, address[]
calldata path, address to, uint deadline)
        external
        returns (uint[] memory amounts);
```

```
    function swapETHForExactTokens(uint amountOut, address[] calldata path, address to, uint deadline)
        external
        payable
        returns (uint[] memory amounts);

    function quote(uint amountA, uint reserveA, uint reserveB) external pure returns (uint amountB);
    function getAmountOut(uint amountIn, uint reserveIn, uint reserveOut) external pure returns (uint amountOut);
    function getAmountIn(uint amountOut, uint reserveIn, uint reserveOut) external pure returns (uint amountIn);
    function getAmountsOut(uint amountIn, address[] calldata path) external view returns (uint[] memory amounts);
    function getAmountsIn(uint amountOut, address[] calldata path) external view returns (uint[] memory amounts);
}
```

2.3 合约接口详细描述

2.3.1 Factory

2.3.1.1 查询型接口

1、getPair

function getPair(address tokenA, address tokenB) external view returns (address pair);

功能描述：通过tokenA 和 tokenB 获取对应的pair地址。如果没有被创建，则返回零地址（0x0000000000000000000000000000000000000000）

参数描述：

参数	类型	描述
tokenA	address	TRC20 token 地址
tokenB	address	TRC20 token 地址

Returns:

address	交易对地址

2、allPairs

function allPairs(uint) external view returns (address pair);

功能描述：返回通过factory合约创建的第n个交易对（0-indexed）的地址，如果对应的索引还没有创建，则返回零地址 (0x0000000000000000000000000000)。

参数描述：

参数	类型	描述
index	uint256	TRC20 token 地址

Returns:

address	交易对地址

3、allPairsLength

function allPairsLength() external view returns (uint);

功能描述：返回通过factory合约创建的所有交易对数量。

Returns:

uint	交易对总数

2.3.1.2 修改型接口

1、createPair

function createPair(address tokenA, address tokenB) external returns (address pair);

功能描述：当tokenA 和 tokenB对应的交易对不存在时，将创建其交易对地址。

参数描述：

参数	类型	描述
tokenA	address	TRC20 token 地址
tokenB	address	TRC20 token 地址

Returns:

address	交易对地址

2.3.1.3 合约事件

1、PairCreated

event PairCreated(address indexed token0, address indexed token1, address pair, uint);

功能描述:每次通过 createPair 创建交易对时发出。

参数描述:

参数	类型	描述
token0	address	TRC20 token 地址
token1	address	TRC20 token 地址
pair	address	justswap中TRC20 token对应的exchange地址
index	uint	对于创建的第一个交易对，最终的 uint 日志值为 1 对于第二个交易对，则为 2，依此类推

2.3.2 Router

2.3.2.1 查询型接口

1、factory

function factory() external pure returns (address);

功能描述:返回 factory 合约地址

Returns:

address	factory 合约地址

2、WETH

function WETH() external pure returns (address);

功能描述:返回 WTRX 合约地址

Returns:

address	WTRX 合约地址

2.3.2.2 修改型接口

1、addLiquidity

```
 function addLiquidity(
  address tokenA,
  address tokenB,
  uint amountADesired,
  uint amountBDesired,
  uint amountAMin,
  uint amountBMin,
  address to,
  uint deadline
) external returns (uint amountA, uint amountB, uint liquidity);
```

功能描述:
为 TRC-20 ⇄ TRC-20 池增加流动性。将 mint出流动性token作为标识。
添加流动性之前需要先进行approve操作。

参数描述:

参数	类型	描述
tokenA	address	TRC20 token 地址
tokenB	address	TRC20 token 地址
amountADesired	uint256	如果B/A价格<= amountBDesired/amountADesired（A贬值），则以此数量作为tokenA的流动性添加。
amountBDesired	uint256	如果A/B价格<= amountADesired/amountBDesired（B贬值），则以此数量作为tokenB的流动性添加。
amountAMin	uint256	根据滑点计算出的范围值，必须小于amountADesired
amountBMin	uint256	根据滑点计算出的范围值，必须小于amountBDesired
to	address	接收 liquidity token 的地址

deadline	uint256	秒的时间戳，超过该时间，交易将revert

Returns:

amountA	uint256	发送到池中的tokenA的数量
amountB	uint256	发送到池中的tokenB的数量
liquidity	uint256	增发给调用者的 liquidity token 数量

2、addLiquidityETH

```
function addLiquidityETH(
  address token,
  uint amountTokenDesired,
  uint amountTokenMin,
  uint amountETHMin,
  address to,
  uint deadline
) external payable returns (uint amountToken, uint amountETH, uint liquidity);
```

功能描述:
使用 TRX 为 TRC-20 ⇄ WTRX 池增加流动性。将 mint 出流动性 token 作为标识。
添加流动性之前需要先进行 approve 操作。

参数描述:

参数	类型	描述
token	address	TRC20 token 地址
amountTokenDesired	uint256	如果 WTRX/token 价格<= msg.value/amountTokenDesired（token贬值），则以此数量作为 token 的流动性添加。
msg.value (amountETHDesired)	uint256	如果 token/WTRX 价格<= amountTokenDesired/msg.value（WTRX贬值），则以此数量作为 WTRX 的流动性添加。
amountTokenMin	uint256	根据滑点计算出的范围值，必须小于 amountTokenDesired

amountETHMin	uint256	根据滑点计算出的范围值，必须小于 msg.value
to	address	接收 liquidity token 的地址
deadline	uint256	秒的时间戳，超过该时间，交易将 revert

Returns:

amountToken	uint256	发送到池中的 token 的数量
amountETH	uint256	TRX 转换为 WTRX 并发送到池中的数量
liquidity	uint256	增发给调用者的 liquidity token 数量

3、removeLiquidity

```
function removeLiquidity(
  address tokenA,
  address tokenB,
  uint liquidity,
  uint amountAMin,
  uint amountBMin,
  address to,
  uint deadline
) external returns (uint amountA, uint amountB);
```

功能描述：
从 TRC-20 ⇄ TRC-20 池移除流动性。移除流动性需要先进行 approve 操作。

参数描述：

参数	类型	描述
tokenA	address	TRC20 token 地址
tokenB	address	TRC20 token 地址
liquidity	uint256	要移除的流动性 token 数量
amountAMin	uint256	根据滑点计算出必须接收到 tokenA 的最小值
amountBMin	uint256	根据滑点计算出必须接收到 tokenB 的最小值
to	address	接收 tokenA/tokenB 的地址

deadline	uint256	秒的时间戳，超过该时间，交易将revert

Returns:

amountA	uint256	收到 tokenA 的数量
amountB	uint256	收到 tokenB 的数量

4、removeLiquidityETH

```
function removeLiquidityETH(
  address token,
  uint liquidity,
  uint amountTokenMin,
  uint amountETHMin,
  address to,
  uint deadline
) external returns (uint amountToken, uint amountETH);
```

功能描述：
从 TRC-20 ⇄ WTRX 池移除流动性，并将 WTRX 转换成 TRX。移除流动性需要先进行 approve 操作。

参数描述：

参数	类型	描述
token	address	TRC20 token 地址
liquidity	uint256	要移除的流动性 token 数量
amountTokenMin	uint256	根据滑点计算出必须接收到 token 的最小值
amountETHMin	uint256	根据滑点计算出必须接收到 TRX 的最小值
to	address	接收 token/TRX 的地址
deadline	uint256	秒的时间戳，超过该时间，交易将revert

Returns:

amountToken	uint256	收到 token 的数量
amountETH	uint256	收到 TRX 的数量

5、removeLiquidityWithPermit

```
function removeLiquidityWithPermit(
  address tokenA,
  address tokenB,
  uint liquidity,
  uint amountAMin,
  uint amountBMin,
  address to,
  uint deadline,
  bool approveMax, uint8 v, bytes32 r, bytes32 s
) external returns (uint amountA, uint amountB);
```

功能描述:
从 TRC-20 ⇄ TRC-20 池移除流动性。由于 permit，移除流动性无需进行 approve 操作。

参数描述:

参数	类型	描述
tokenA	address	TRC20 token 地址
tokenB	address	TRC20 token 地址
liquidity	uint256	要移除的流动性 token 数量
amountAMin	uint256	根据滑点计算出必须接收到 tokenA 的最小值
amountBMin	uint256	根据滑点计算出必须接收到 tokenB 的最小值
to	address	接收 tokenA/tokenB 的地址
deadline	uint256	秒的时间戳，超过该时间，交易将revert
approveMax	bool	签名中 approve 的数量是否为 uint(-1) 或者 liquidity
v	uint8	permit 签名中的 v 组件
r	bytes32	permit 签名中的 r 组件
s	bytes32	permit 签名中的 s 组件

Returns:

amountA	uint256	收到 tokenA 的数量

amountB	uint256	收到 tokenB 的数量

6、removeLiquidityETHWithPermit

```
function removeLiquidityETHWithPermit(
  address token,
  uint liquidity,
  uint amountTokenMin,
  uint amountETHMin,
  address to,
  uint deadline,
  bool approveMax, uint8 v, bytes32 r, bytes32 s
) external returns (uint amountToken, uint amountETH);
```

功能描述:
从 TRC-20 ⇄ WTRX 池移除流动性，并将 WTRX 转换成 TRX。由于 permit，移除流动性无需进行 approve 操作。

参数描述:

参数	类型	描述
token	address	TRC20 token 地址
liquidity	uint256	要移除的流动性 token 数量
amountTokenMin	uint256	根据滑点计算出必须接收到 token 的最小值
amountETHMin	uint256	根据滑点计算出必须接收到 TRX 的最小值
to	address	接收 token/TRX 的地址
deadline	uint256	秒的时间戳，超过该时间，交易将revert
approveMax	bool	签名中 approve 的数量是否为 uint(-1) 或者 liquidity
v	uint8	permit 签名中的 v 组件
r	bytes32	permit 签名中的 r 组件
s	bytes32	permit 签名中的 s 组件

Returns:

amountToken	uint256	收到 token 的数量

amountETH	uint256	收到 TRX 的数量

7、removeLiquidityETHSupportingFeeOnTransferTokens

```
function removeLiquidityETHSupportingFeeOnTransferTokens(
  address token,
  uint liquidity,
  uint amountTokenMin,
  uint amountETHMin,
  address to,
  uint deadline
) external returns (uint amountETH);
```

功能描述:
从 TRC-20 ⇄ WTRX 池移除流动性，并将 WTRX 转换成 TRX。此函数用于收取转账费用（或通缩型）的 token 。移除流动性需要先进行 approve 操作。

参数描述:

参数	类型	描述
token	address	TRC20 token 地址
liquidity	uint256	要移除的流动性 token 数量
amountTokenMin	uint256	根据滑点计算出必须接收到 token 的最小值
amountETHMin	uint256	根据滑点计算出必须接收到 TRX 的最小值
to	address	接收 token/TRX 的地址
deadline	uint256	秒的时间戳，超过该时间，交易将revert

Returns:

amountETH	uint256	收到 TRX 的数量

8、removeLiquidityETHWithPermitSupportingFeeOnTransferTokens

```
function removeLiquidityETHWithPermitSupportingFeeOnTransferTokens(
  address token,
  uint liquidity,
```

```
  uint amountTokenMin,
  uint amountETHMin,
  address to,
  uint deadline,
  bool approveMax, uint8 v, bytes32 r, bytes32 s
) external returns (uint amountETH);
```

功能描述:
从 TRC-20 ⇄ WTRX 池移除流动性，并将 WTRX 转换成 TRX。用于收取转账费用的 token 。由于 permit, 移除流动性无需进行 approve 操作。

参数描述:

参数	类型	描述
token	address	TRC20 token 地址
liquidity	uint256	要移除的流动性 token 数量
amountTokenMin	uint256	根据滑点计算出必须接收到 token 的最小值
amountETHMin	uint256	根据滑点计算出必须接收到 TRX 的最小值
to	address	接收 token/TRX 的地址
deadline	uint256	秒的时间戳，超过该时间，交易将revert
approveMax	bool	签名中 approve 的数量是否为 uint(-1) 或者 liquidity
v	uint8	permit 签名中的 v 组件
r	bytes32	permit 签名中的 r 组件
s	bytes32	permit 签名中的 s 组件

Returns:

amountETH	uint256	收到 TRX 的数量

9、swapExactTokensForTokens

```
function swapExactTokensForTokens(
  uint amountIn,
  uint amountOutMin,
  address[] calldata path,
  address to,
```

```
  uint deadline
) external returns (uint[] memory amounts);
```

功能描述：
将精准数量的输入 token 兑换成尽可能多的输出 token。参数 path 的第一个原素是输入 token 地址，最后一个原素是输出 token 地址，任何中间的原素代表要交易的中间交易对。兑换之前需要先进行 approve 操作。

参数描述：

参数	类型	描述
amountIn	uint256	输入 token 的数量
amountOutMin	uint256	根据滑点计算出必须接收到输出 token 的最小值
path	address[] calldata	一组 token 地址，path.length 必须>=2。所有相邻 token 组成的交易池必须存在并有流动性
to	address	接收输出 token 的地址
deadline	uint256	秒的时间戳，超过该时间，交易将revert

Returns:

amounts	uint256[] memory	输入 token 和所有后续输出 token 的数量

10、swapExactETHForTokens

```
function swapExactETHForTokens(
  uint amountOutMin,
  address[] calldata path,
  address to,
  uint deadline
) external payable returns (uint[] memory amounts);
```

功能描述：
将精准数量的 TRX 兑换成尽可能多的输出 token。参数 path 的第一个元素是 WTRX 地址，最后一个元素是输出 token 地址，任何中间的元素代表要交易的中间交易对。
参数描述：

参数	类型	描述
msg.value	uint256	发送 TRX 的数量
amountOutMin	uint256	根据滑点计算出必须接收到输出 token 的最小值
path	address[] calldata	一组 token 地址，path.length 必须>=2。所有相邻 token 组成的交易池必须存在并有流动性
to	address	接收输出 token 的地址
deadline	uint256	秒的时间戳，超过该时间，交易将revert

Returns:

amounts	uint256[] memory	输入 token 和所有后续输出 token 的数量

11、swapTokensForExactETH

```
function swapTokensForExactETH(
  uint amountOut,
  uint amountInMax,
  address[] calldata path,
  address to,
  uint deadline
) external returns (uint[] memory amounts);
```

功能描述：
将尽可能少的输入 token 兑换成精准数量的 TRX。参数 path 的第一个元素是输入 token 地址，最后一个元素是 WTRX 地址，任何中间的原素代表要交易的中间交易对。兑换之前需要先进行 approve 操作。如果 to 地址是智能合约，它必须具有接收 TRX 的能力。

参数描述：

参数	类型	描述
amountOut	uint256	要接收的 TRX 数量
amountInMax	uint256	根据滑点计算出必须支付的输入 token 的最大值

path	address[] calldata	一组 token 地址，path.length 必须>=2。所有相邻 token 组成的交易池必须存在并有流动性
to	address	接收 TRX 的地址
deadline	uint256	秒的时间戳，超过该时间，交易将revert

Returns:

amounts	uint256[] memory	输入 token 和所有后续输出 token 的数量

12、swapExactTokensForETH

```
function swapExactTokensForETH(
  uint amountIn,
  uint amountOutMin,
  address[] calldata path,
  address to,
  uint deadline
) external returns (uint[] memory amounts);
```

功能描述：
将精准数量的输入 token 兑换成尽可能多的 TRX。参数 path 的第一个元素是输入 token 地址，最后一个元素是 WTRX 地址，任何中间的原素代表要交易的中间交易对。如果 to 地址是智能合约，它必须具有接收 TRX 的能力。

参数描述：

参数	类型	描述
amountIn	uint256	输入 token 的数量
amountOutMin	uint256	根据滑点计算出必须输出 token 的最小值
path	address[] calldata	一组 token 地址，path.length 必须>=2。所有相邻 token 组成的交易池必须存在并有流动性
to	address	接收 TRX 的地址
deadline	uint256	秒的时间戳 ,超过该时间，交易将revert

Returns:

amounts	uint256[] memory	输入 token 和所有后续输出 token 的数量

13、swapETHForExactTokens

```
function swapETHForExactTokens(
  uint amountOut,
  address[] calldata path,
  address to,
  uint deadline
) external payable returns (uint[] memory amounts);
```

功能描述:
将尽可能少的输入 token 兑换成精准数量的 TRX。参数 path 的第一个元素是 WTRX 地址，最后一个元素是输出 token 地址，任何中间的原素代表要交易的中间交易对。 如果 to 地址是智能合约，它必须具有接收 TRX 的能力。

参数描述:

参数	类型	描述
amountOut	uint256	要接收的输出 token 数量
msg.value (amountInMax)	uint256	根据滑点计算出必须支付的 TRX 的最大值
path	address[] calldata	一组 token 地址，path.length 必须>=2。所有相邻 token 组成的交易池必须存在并有流动性
to	address	接收输出 token 的地址
deadline	uint256	秒的时间戳，超过该时间，交易将revert

Returns:

amounts	uint256[] memory	输入 token 和所有后续输出 token 的数量

14、swapExactTokensForTokensSupportingFeeOnTransferTokens

```
function swapExactTokensForTokensSupportingFeeOnTransferTokens(
  uint amountIn,
  uint amountOutMin,
  address[] calldata path,
  address to,
  uint deadline
) external;
```

功能描述：
将精准数量的输入 token 兑换成尽可能多的输出 token。参数 path 的第一个原素是输入 token 地址，最后一个原素是输出 token 地址，任何中间的原素代表要交易的中间交易对。用于收取转账费用的 token 。兑换之前需要先进行 approve 操作。

参数描述：

参数	类型	描述
amountIn	uint256	输入 token 的数量
amountOutMin	uint256	根据滑点计算出必须接收到输出 token 的最小值
path	address[] calldata	一组 token 地址，path.length 必须>=2。所有相邻 token 组成的交易池必须存在并有流动性
to	address	接收输出 token 的地址
deadline	uint256	秒的时间戳，超过该时间，交易将revert

15、swapExactETHForTokensSupportingFeeOnTransferTokens

```
function swapExactETHForTokensSupportingFeeOnTransferTokens(
  uint amountOutMin,
  address[] calldata path,
  address to,
  uint deadline
) external payable;
```

功能描述：
将精准数量的 TRX 兑换成尽可能多的输出 token。参数 path 的第一个原素是 WTRX 地址，最后一个元素是输出 token 地址，任何中间的元素代表要交易的中间交易对。本函数用于收取转账费用（或通缩型）的 token 。

参数描述:

参数	类型	描述
msg.value (amountIn)	uint256	发送 TRX 的数量
amountOutMin	uint256	根据滑点计算出必须接收到输出 token 的最小值
path	address[] calldata	一组 token 地址，path.length 必须>=2。所有相邻 token 组成的交易池必须存在并有流动性
to	address	接收输出 token 的地址
deadline	uint256	秒的时间戳，超过该时间，交易将revert

16、swapExactTokensForETHSupportingFeeOnTransferTokens

```
function swapExactTokensForETHSupportingFeeOnTransferTokens(
  uint amountIn,
  uint amountOutMin,
  address[] calldata path,
  address to,
  uint deadline
) external;
```

功能描述:
将精准数量的输入 token 兑换成尽可能多的 TRX。参数 path 的第一个元素是输入 token 地址，最后一个元素是 WTRX 地址，任何中间的原素代表要交易的中间交易对。此函数用于收取转账费用（或通缩型）的 token 。如果 to 地址是智能合约，它必须具有接收 TRX 的能力。

参数描述:

参数	类型	描述
amountIn	uint256	输入 token 的数量
amountOutMin	uint256	根据滑点计算出必须输出 token 的最小值
path	address[] calldata	一组 token 地址，path.length 必须>=2。所有相邻 token 组成的交易池必须存在并有流动性

to	address	接收 TRX 的地址
deadline	uint256	秒的时间戳，超过该时间，交易将revert